LINUX SYSTEM ADMINISTRATION

Crontab Task Automation and Instagram Photo Backup

Practical Assignment — Questions with Answers

Prepared by

Sujal Chouhan

Topics: Cron scheduling • Real-life automation • Shell scripting • Date-based backups • Logging • Verification with tail

This is a practical guide and sample report. Run the commands on your own Linux system and include genuine command output for proof. Replace /home/yourusername with your actual home directory.

Objective

Identify useful cron automation scenarios, create a daily scheduled playlist update, write a date-stamped Instagram photo backup script, schedule it weekly with logging, and verify the log output.

Question 1 — Three real-life crontab scenarios

Q. List three real-life scenarios where crontab automation is useful and explain one benefit of each.

Answer

```
1. Daily database backup — creates regular backups automatically and reduces the risk of data loss.
2. Log cleanup/rotation — removes or archives old logs on a schedule to save disk space.
3. System health report — periodically records disk usage, memory, and service status so problems can be spotted ear
```

Question 2 — Run update_playlist.sh daily at 2:00 AM

Q. Write the crontab entry and explain each field.

```
0 2 * * * /home/yourusername/update_playlist.sh
```

Field meanings: **0** = minute 0; **2** = hour 2 (2 AM); first ***** = every day of month; second ***** = every month; last ***** = every day of week. Ensure the script exists and is executable: **chmod +x /home/yourusername/update_playlist.sh**. Edit the current user's cron table with **crontab -e**.

Question 3 — Create backup_instagram_photos.sh

Q. Create a script that copies all files from InstagramPhotos to a backup folder named with today's date.

Answer / Script

```
#!/bin/bash
set -euo pipefail

SOURCE="/home/yourusername/InstagramPhotos"
BACKUP_ROOT="/home/yourusername/Backups"
TODAY="$(date +%F)"
DEST="$BACKUP_ROOT/InstagramPhotos_$TODAY"

mkdir -p "$DEST"
if [[ ! -d "$SOURCE" ]]; then
  echo "ERROR: Source folder not found: $SOURCE"
  exit 1
fi

cp -a "$SOURCE"/. "$DEST"/
echo "$(date '+%F %T') Backup completed: $SOURCE -> $DEST"
```

Save the script as /home/yourusername/backup_instagram_photos.sh. Replace yourusername with your real Linux username. The date command creates a YYYY-MM-DD folder name. cp -a copies files and preserves common file attributes; "$SOURCE"/. also includes hidden files. The script logs a success line only after the copy succeeds.

```
chmod +x /home/yourusername/backup_instagram_photos.sh
mkdir -p /home/yourusername/InstagramPhotos
mkdir -p /home/yourusername/Backups
```

Question 4 — Run every Sunday at 11:30 PM and log output

Q. Schedule the backup every Sunday at 11:30 PM and redirect stdout and stderr to ~/backup_log.txt.

```
30 23 * * 0 /home/yourusername/backup_instagram_photos.sh >> /home/yourusername/backup_log.txt 2>&1
```

Field meanings: **30** = minute 30; **23** = 11 PM in 24-hour time; ***** = every day of month; ***** = every month; **0** = Sunday on most cron implementations. **>> backup_log.txt** appends standard output to the log, and **2>&1** sends standard error to the same log. Add this line using **crontab -e** for the user who owns the files.

Before relying on cron, test the script manually and inspect the current cron table:

```
/home/yourusername/backup_instagram_photos.sh
crontab -e
crontab -l
```

Question 5 — Show the last five log lines as proof

Q. After the backup runs, how do you view and submit the last five lines of backup_log.txt?

Answer / Commands

```
tail -n 5 /home/yourusername/backup_log.txt

# Optional: add a timestamped test run to the same log
/home/yourusername/backup_instagram_photos.sh >> /home/yourusername/backup_log.txt 2>&1
tail -n 5 /home/yourusername/backup_log.txt
```

Use the first command after the scheduled job has run to capture the actual last five lines. The optional manual run is useful for testing immediately, but it does not prove the Sunday schedule itself has fired. Do not submit invented sample output as real proof.

Example of what a successful log line may look like

```
2026-10-04 23:30:00 Backup completed: /home/yourusername/InstagramPhotos -> /home/yourusername/Backups/InstagramPhot
```

This is an illustrative example only; your actual timestamp and folder date will depend on when you run the script.

Final checklist

```
[ ] Replace every /home/yourusername path with your real home path.
[ ] Save the script and run chmod +x on it.
[ ] Run the script manually and verify the dated backup folder exists.
[ ] Add the Sunday cron entry with crontab -e and confirm with crontab -l.
[ ] After execution, capture tail -n 5 ~/backup_log.txt as genuine evidence.
```

Prepared by: Sujal Chouhan